

Stage 2 实验报告

2022011223 熊泽恩

实验内容

为了完成这次实验，我进行了以下工作：

- 修改了 `utils/namer.py`，补充了 `visitIdentifier`、`visitDeclaration` 和 `visitAssignment`，使其能够正常生成 AST。
 - `visitIdentifier` 函数处理的是标识符，使用 `ctx.lookup` 在当前作用域中查找标识符对应的符号，如果标识符没有被声明，就抛出异常。
 - `visitDeclaration` 函数处理的是声明语句，使用 `ctx.lookup` 在当前作用域中查找是否有同名变量已经被声明，如果已经被声明就抛出异常。
 - `visitAssignment` 函数处理的是赋值语句，会检查赋值式的左侧是否为标识符，如果不是则抛出异常。
- 修改了 `frontend/tacgen/tacgen.py`，补充了 `visitIdentifier`、`visitDeclaration` 和 `visitAssignment`，使其能够正常生成中间代码。
 - `visitIdentifier` 函数处理的是标识符，它将标识符映射到一个临时变量，以便在后续的代码生成中使用。
 - `visitDeclaration` 函数处理的是声明语句，它为每个新声明的变量分配一个临时变量；如果有初始值，则也要赋初值。
 - `visitAssignment` 函数处理的是赋值语句，它生成赋值指令的中间代码，将右侧表达式的值赋给左侧变量。
- 修改了 `backend/riscvasmemitter.py`，重载了 `RiscvAsmEmitter.RiscvInstrSelector.visitAssign`，使其能够用 `mv` 指令，将赋值语句翻译为目标代码。

Stage 2 思考题

Step 5

第 1 题

我们假定当前栈帧的栈顶地址存储在 `sp` 寄存器中，请写出一段 **RISC-V 汇编代码**，将栈帧空间扩大 16 字节。（提示 1：栈帧由高地址向低地址延伸；提示 2：RISC-V 汇编中 `addi reg0, reg1, <立即数>` 表示将 `reg1` 的值加上立即数存储到 `reg0` 中。）

```
1 | addi sp, sp, -16
```

第 2 题

有些语言允许在同一个作用域中多次定义同名的变量，例如这是一段合法的 Rust 代码（你不需要精确了解它的含义，大致理解即可）：

```
1 fn main() {  
2     let a = 0;  
3     let a = f(a);  
4     let a = g(a);  
5 }
```

其中 `f(a)` 中的 `a` 是上一行的 `let a = 0;` 定义的, `g(a)` 中的 `a` 是上一行的 `let a = f(a);`。

如果 MiniDecaf 也允许多次定义同名变量, 并规定新的定义会覆盖之前的同名定义, 请问在你的实现中, 需要对定义变量和查找变量的逻辑做怎样的修改? (提示: 如何区分一个作用域中**不同位置**的变量定义?)

如果 MiniDecaf 也允许多次定义同名变量, 则符号表中关于变量名的映射应该从单一的 `VarSymbol` 对象改为一个列表 `List[VarSymbol]`, 这样可以存储同一变量名的多个定义。

- 在定义新变量时, 应该在当前作用域的符号表中对应的列表末尾添加一个新的 `VarSymbol` 对象。
- 查找变量时, 应该从当前作用域的符号表中对应的列表中查找, 并返回列表中最后一个元素, 即最新定义的 `VarSymbol`。